CAS CORE 8: A COMPATIBILITY AND EVIDENCE KERNEL FOR AUDITABLE COGNITIVE-ARCHITECTURE SOFTWARE

LITTLE HENDERSON

Abstract. CAS Core 8 is a small, auditable compatibility and evidence kernel for the Cognitive Architecture System (CAS) software family. It is deliberately not a rewrite of CAS v7.7 and is not installed on any production route. Its purpose is narrower and, we argue, prior: before any v8 semantics can be trusted, the platform must make source identity, semantic namespace, compatibility profile, registry integrity, action intent, and claim boundaries explicit enough to test. The kernel contributes five mechanisms: structural semantic references that keep TKS and CAS namespaces distinct and force positional or historical correspondences to be typed as translations, alignments, or hypotheses—never silently as identities; a research-boundary object whose ring, license, and disclosure fields must be jointly consistent and are rechecked rather than trusted; deterministic canonical evidence encoding with exact-decimal tagging, cycle-awareness, and single-route serialization; a hash-verified, process-isolated legacy compiler compatibility profile that executes the exact active CAS compiler bytes in a sanitized worker and demands fresh replay before any structural decision passes; and exact-byte-backed claim and theorem registries whose loaders fail closed on any structural irregularity. A central design commitment runs through all five: *structural consistency is not authorization.* In this candidate, authorization is always denied; claim-bearing intent is refused even when a locally constructed ledger says "licensed," because the v0.1 kernel has no authenticated reviewer or legal trust root. Formal conformance, v8 semantic migration, claim authority, and independent review remain explicitly open.

1. Introduction

Two recurring failure classes in long-lived research software motivate the kernel.

Semantic drift. The TKS/CAS program spans multiple ontologies—TKS Foundations and Worlds; CAS Drives, Domains, Registers, and States; Noetic Topos objects—whose elements are related by history and position but are not interchangeable. Prior audits repeatedly found positional relabels treated as paraphrases and cross-namespace conflations voiding otherwise sound work. Untyped string identifiers make such drift invisible to tooling.

Unauthorized claim promotion. A green test, a self-computed hash, or a generated receipt is structural evidence about software behavior. Nothing in such artifacts establishes reviewer identity, legal authority, or empirical validity—yet historical AI-authored, self-hashed "licensing receipts" demonstrated how easily structural artifacts masquerade as authority. A kernel that cannot distinguish *consistent* from *authorized* will eventually launder one into the other.

Date: August 7, 2026.

White paper v0.1, 2026-08-07. Artifact status: DRAFT / OPEN / RING 4 / NOT LICENSED / HELD. Source basis: `CAS_CORE_V8_CANDIDATE_SPEC_v0.1.md` (SHA-256 `5FE6F91D...`); implementation `src/cas_core_v8/`; Gate 4 receipt: 61 test methods, test-ID SHA-256 `E2AF5014....` Drafting this paper promotes no claim.

CAS Core 8 addresses both by making identity structural and authorization impossible by construction in v0.1—there is no code path that grants it—with future claim authorization gated on a separately reviewed design.

2. Semantic identity

Definition 2.1 (Semantic reference)**.** A semantic reference is the structural tuple

$$(\text{namespace},\ \text{local_id},\ \text{version},\ \text{source_id},\ \text{definition_sha256}).$$

TKS Foundation, TKS World, CAS Drive, CAS Domain, CAS Register, CAS State, and Noetic Topos object identities occupy different namespaces.

Rule 2.2 (No silent identity)**.** A positional or historical correspondence between references is represented as TRANSLATION, ALIGNMENT, or HYPOTHESIS; it is never silently represented as IDENTITY.

Rule 2.2 encodes, at the type level, the canon rule that positional relabels (for example, between TKS Foundations and CAS Drive names) are not paraphrases and must not be conflated.

Invariant 2.3 (Injective rendering)**.** *Rendered identifiers are injective: delimiter-containing components cannot create the same identifier as a different structural tuple. Equality-overriding subclasses are not admitted as identity-bearing public values; exact types and primitive fields determine every boundary and semantic identity check.*

3. The research boundary object

Definition 3.1 (Research boundary)**.** Every candidate artifact carries a boundary. Ring 4 enforces all of: admission state OPEN; product state NOT LICENSED; and public disclosure *false*. Contradictory combinations are invalid at construction.

Invariant 3.2 (Recheck, do not trust)**.** *Public operations derive and recheck the boundary rather than trusting a shared mutable object or a recomputable self-hash. A structural receipt or green test does not alter the boundary. A self-digest detects mutation after construction but is not a signature, reviewer identity, or authority grant.*

4. Canonical evidence encoding

Byte identity and comparison identity are recorded separately: raw bytes and the declared CRLF-to-LF comparison identity coexist, with lone-CR, CRLF, and lone-LF counts never conflated. The evidence encoding

(1) is deterministic UTF-8 JSON with sorted string keys;
(2) rejects unordered containers, binary floats, NaN, and infinity;
(3) tags exact decimals so Decimal(`"1.0"`) cannot collide with the string `"1.0"`; and
(4) gives each public type exactly one declared serialization route.

Canonicalization is bounded and cycle-aware. Review-visible machine identifiers use declared normalized/ASCII profiles; unrestricted prose Unicode remains separate. These rules exist because evidence digests are load-bearing: any nondeterminism or type pun in the encoder silently invalidates every downstream hash comparison.

5. Legacy compiler compatibility profile

The only admitted legacy profile in v0.1 binds the exact active Python compiler bytes listed in the Gate 4 record. It is explicitly `legacy_only` and produces no CAS v8 semantic translation.

Rule 5.1 (Deterministic closure)**.** The profile: (1) verifies the requested source hash, then reads and hashes compiler bytes *before* execution; (2) forces the Python-only engine profile and an exact frozen policy; (3) rejects `use` imports, because no import-DAG manifest is yet admitted; (4) executes the verified bytes in a fresh `python -I -S -B` worker with a sanitized environment rather than inheriting parent module state; (5) samples and binds the worker runtime before compiler code executes, without WMI or subprocess probing; (6) serializes cooperating workers through a profile-bound advisory lock (90-second acquisition bound, 10 ms poll, 10 ms post-release yield) without retrying a failed worker or claiming lock fairness; (7) records raw and LF compiler identities, interpreter and worker bytes, import search paths, loaded-module inventory, exact syntax/semantic/engine profiles, result digest, diagnostics, and the research boundary; and (8) requires fresh replay against the same source and compiler identity before a structural decision can pass.

Remark 5.2 (Isolation is not a sandbox). This is Python-process isolation, *not* an operating-system sandbox. Host native libraries and the operating system are not content-addressed, and every receipt exposes both limitations as *false* capability fields. Worker transport failures record only exit code, stdout/stderr byte counts, and digests; an empty or malformed response fails closed and is never retried. If two successful workers disagree on dependency profiles, the error records both profile hashes and at most 32 keyed difference paths, with environment values represented only by hashes.

Rule 5.3 (Fractal bridge)**.** Legacy fractal digit sequences require an explicit caller-supplied catalog of typed finite arrows from the TKS v8 kernel; composition uses semigroupoid paths, and missing digits or source/target mismatches fail closed. The bridge proves only type-correct composition in the finite reference kernel—not the meaning of a legacy digit.

6. Registry integrity

The claim and theorem registries are exact-byte-backed snapshots validated against the Project Coherence record contracts. Parsed headers, schemas, and records are reconstructed from and rechecked against the retained raw ledger and schema bytes; a direct constructor or dataclass copy cannot preserve an old raw identity while substituting parsed content. The loader rejects duplicate JSON keys, unknown root or record fields, wrong enums or shapes, invalid identifiers, ambiguous versions, and missing required metadata. Loaded values are recursively immutable, and each selected record has a deterministic digest bound to the registry bytes.

Invariant 6.1 (Metadata is not authority)**.** *The text* LICENSED *inside a record is necessary metadata, not sufficient authority. The v0.1 candidate has no authenticated reviewer or legal trust root and therefore cannot authorize a claim-bearing action.*

7. Runtime decision model

Action operation, target, and effect class are explicit enums. Nonclaim research evidence may pass only a structural replay comparison, and only when every exact compatibility field and digest validates. Two invariants are absolute in v0.1.

Invariant 7.1 (No authorization path)**.** *The fields* `authorization_granted` *and the compatibility spelling* `allowed` *are always false; and claim-bearing intent is denied even if a locally constructed ledger says* LICENSED*. A self-hash or Python-local sentinel is not issuance provenance.*

Future claim authorization requires a separately reviewed design binding: an exact trusted registry/schema manifest; an authenticated authority and trust root; reviewer and admission receipts; Evidence Packet identities and verification status; claim ID and exact version; and the code, model, environment, semantic, and calibration identities of the requested operation.

8. Relation to companion candidates

CAS Core 8 is the shared substrate of the Gate 4 engineering lane. *CAS PCBA 8* consumes the semantic-identity, tagged-value, canonical-evidence, and research-boundary contracts for its board model, design-rule checks, and traces. *CAS-Ohm 1.0* consumes the same evidence discipline for typed quantities, calibration profiles, and fail-closed research packets. *TKS v8.0* supplies the finite typed-arrow kernel referenced by the fractal bridge. *Noetic Topos 2.0* supplies the ambient-versus-resolved evidence distinction that the kernel's records anticipate: a marked cover proves no measurement true.

At the bound Gate 4 receipt, the focused CAS Core v8 suite records 61 passing test methods within an integrated three-suite run of 208 methods and 272 parameterized subtests—historical, source-bound software receipts, not experiments or authority.

9. Explicitly open obligations

(1) A hermetic subprocess and toolchain bundle is not established by Python-level isolation alone.
(2) Python/Rust conformance is not established; profiles remain separate.
(3) No v8 parser or full semantic migration is claimed.
(4) No claim-authority implementation is admitted.
(5) Independent software, formal-methods, semantics, and product-safety review is still required.

10. Conclusion

CAS Core 8's thesis is that trustworthy cognitive-architecture software begins with an evidence kernel that cannot be talked into overclaiming: identities that are structural, boundaries that are rechecked, encodings that are deterministic, legacy execution that is hash-verified and replayed, registries that are byte-backed, and an authorization path that in v0.1 has no code path to "yes." Everything a future CAS v8 wants to assert—semantics, conformance, claims—must be *added* to this kernel through reviewed gates, rather than subtracted from an initially permissive system. That inversion is the contribution.

References

[1] Project Coherence, Gate 4 record and artifact manifest v0.1 (54 hash-bound artifacts; suite identities).
[2] Project Coherence record contracts (claim and theorem registry schemas).
[3] Companion white papers in this series, v0.1: *TKS v8.0*; *Noetic Topos 2.0*; *CAS PCBA 8*; *CAS-Ohm 1.0*.